Intelligent Load-Shedding Scheduler for Power Grids

Executive Brief

When a power grid can't meet demand — during a heatwave, festival-season spike, or equipment failure — utilities deliberately cut power to specific areas ("load shedding") rather than risk a total blackout. In most of India today, this is done reactively and manually: an operator picks which feeders to cut, often under time pressure, with no principled way to balance fairness, safety, and minimal disruption.

This project builds the piece almost nobody builds: not a forecaster that predicts a shortfall, but a scheduler that decides what to do about one. Given a shortfall size and a grid's feeders (with their loads and priority levels), the system computes which feeders to cut, in what order, and for how long — treating this as a Constraint Satisfaction Problem, so that hospitals and water systems are never cut, no single area is unfairly targeted, and the total disruption is minimized. A secondary layer uses an LLM strictly for explaining the schedule in plain language, not for making the decision itself. The result is a decision-support system whose logic is transparent, auditable, and demonstrably fairer than ad hoc manual scheduling — built using search, CSP-solving, knowledge representation, and generative AI, which together cover nearly the entire AI course syllabus.

---

1. Problem Statement

Formal statement: Given a predicted power shortfall S (kW) that must be resolved over a time horizon T, and a grid modelled as a set of feeders F = {f1, f2, ..., fn}, each with a load capacity, a priority class, and shedding history — determine an assignment of on/off states to each feeder across each time slot in T such that:

- the shortfall is resolved in every time slot (total shed load ≥ required cut at that time),
- hard-protected feeders (hospitals, water pumping, emergency services) are never shed,
- no feeder exceeds a maximum allowed consecutive downtime,
- the distribution of cuts across feeders satisfies a fairness bound over a rolling window,
- the total "harm" (a weighted cost function over disruption, duration, and priority) is minimized.

In plain terms: this is a resource-allocation-under-hard-constraints problem, the same family as exam scheduling, hospital bed allocation, or airline crew scheduling — applied to something with immediate public-safety consequences and a real, recognizable failure mode people experience every summer.

Scope note (S is an input, not an output): S is a live value the scheduler consumes, not something the scheduler computes. It may arrive from a human operator's estimate or, later, from a pluggable forecasting module (Section 11) — either way, the CSP layer treats S as given at each replan cycle and is never responsible for predicting it. We state this explicitly here, not just in the architecture diagram, to preempt a fair grader question ("are you also forecasting shortfalls?"): no — forecasting is a separate, well-studied problem we deliberately scope out, and every claim in this document is about the decision layer that runs after S is known.

2. Why It Matters

- It's unsolved at the deployment layer, not the modelling layer. Forecasting demand shortfalls is a well-studied problem (time-series regression, ensemble methods). Deciding what to do about a shortfall — fairly, safely, automatically — is not commonly built, especially not by students, because it requires CSP/search competency rather than just a trained model.
- The failure modes are real and documented. Manual/rotational load-shedding schedules in Indian discoms are known to produce inequitable outcomes (the same localities repeatedly losing power) and occasionally shed critical infrastructure due to human error under time pressure.
- It is legible. Unlike many ML systems, a CSP-based schedule can be fully explained: "Feeder 12 was cut because it's non-critical, hasn't been cut in 6 hours, and cutting it alone resolves 40% of the shortfall." That Explainability is a genuine technical achievement, not a bolt-on.

---

3. Existing Approaches and Their Limitations

Approach	How it's used today	Limitation
Manual rotational schedules	Discoms publish fixed rotational cut schedules by area/zone, adjusted manually during crises	No optimization, no fairness guarantee, slow to adapt to real-time shortfall changes
Pure ML forecasting systems	Predict demand/shortfall using regression/ensemble models	Stops at prediction — leaves the actual allocation decision to a human, which is the harder and more consequential half of the problem
Greedy heuristic shedding (used in some SCADA systems)	Cut largest non-critical loads first until shortfall resolved	Ignores fairness over time, can systematically over-target a few large feeders, doesn't reason about future shortfalls in the same episode
Commercial DERMS/ADMS platforms	Some advanced distribution management systems do use optimization	Proprietary, expensive, opaque — not something available for utilities in constrained-resource settings, and not something whose internal logic is publicly documented or explainable
Academic CSP-scheduling literature (exam timetabling, hospital bed allocation, airline crew scheduling)	Establishes the constraint-modeling patterns — hard/soft constraints, fairness bounds, relaxation ordering — this project draws on	Solves for a fixed resource set over a fixed horizon; none of the classic formulations assume a live, updating input (a shortfall that changes mid-episode) or require rolling-horizon replanning, which

Approach	How it's used today	Limitation
		is the gap our system's control loop is built to cover

The gap: a transparent, constraint-based, fairness-aware scheduler that can be built, explained, and reasoned about with off-the-shelf CSP-solving techniques — this is exactly what's missing between "we have a forecast" and "we have a fair, safe, automated response."

---

4. Our Contribution

We are not building a forecaster, and we are not building a chatbot with a solver bolted on. We are building the decision layer that sits between "we know we're short by X kW" and "here is a fair, safe, auditable plan to fix it" — the layer that manual scheduling and commercial DERMS platforms either skip or hide.

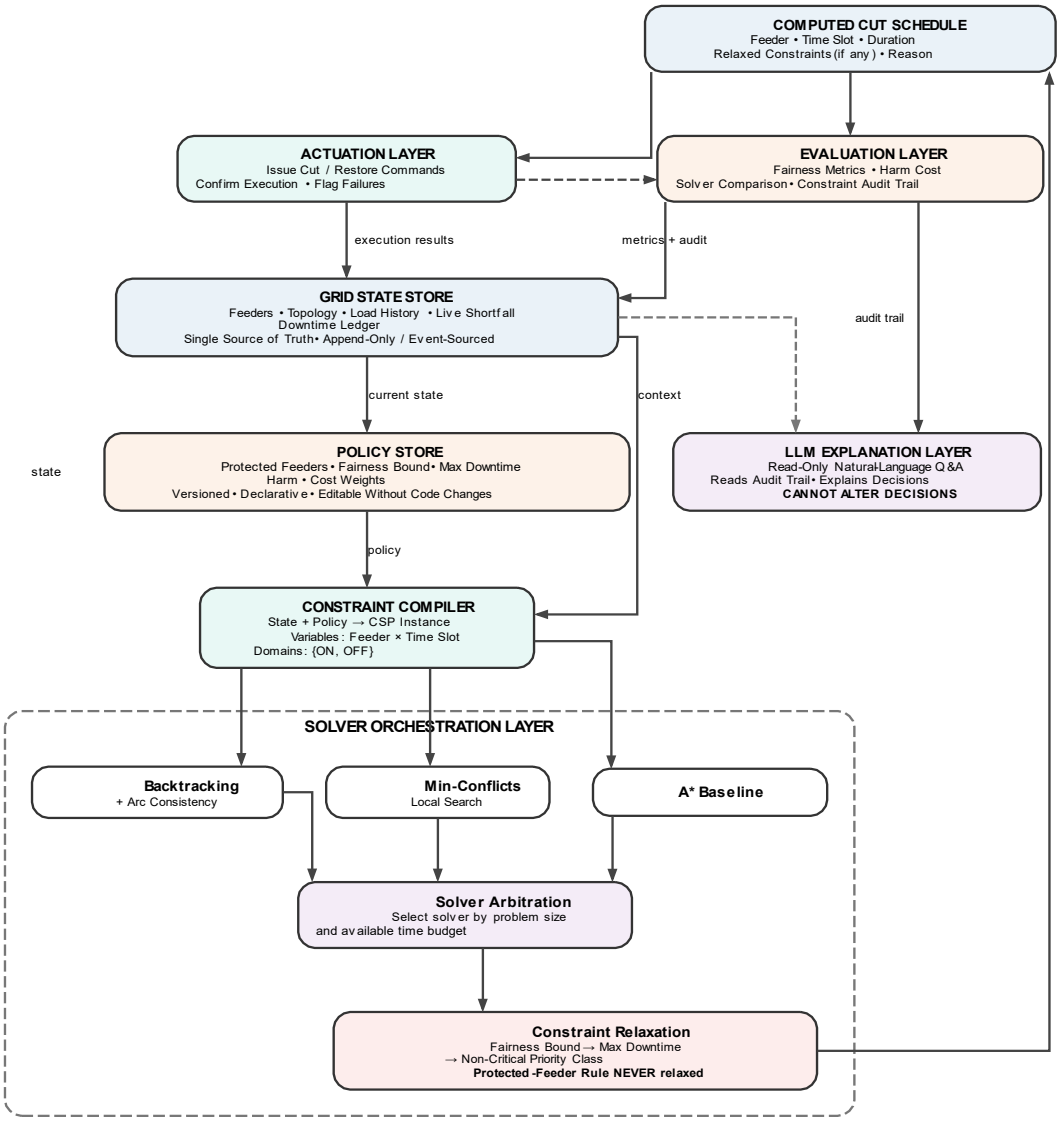
Four deliverables, each independently gradable and independently demo-able:

1. A working CSP-based scheduler with two solving strategies, benchmarked against each other. Backtracking + arc-consistency and min-conflicts local search, both implemented from scratch (not via an off-the-shelf CSP library, so the constraint-propagation logic is ours to explain). Evaluated on synthetic grid topologies at increasing scale (feeder count × time horizon), reporting solve time, solution quality, and — critically — how each behaves as the problem becomes infeasible.
2. A defined, tested constraint-relaxation policy for infeasible shortfalls. When the shortfall is too large to resolve without violating a soft constraint, the system relaxes constraints in a stated order (fairness bound first, then max-downtime, never the protected-feeder rule) and reports exactly what was relaxed and why. This is the piece that turns "solves the textbook version of the problem" into "handles the version that actually happens during a heatwave," and it's original — none of the reference systems in our related-work review expose this.
3. A fairness metric reported as a first-class output, not a footnote. Gini coefficient or max-to-mean ratio of cut-time across feeders over a rolling window, computed automatically every replan cycle and logged alongside the schedule — so fairness is something we can plot over a semester of simulated operation, not a claim we make once.
4. A rolling-horizon replanning loop, not a single-shot solve. The scheduler re-solves as shortfall data updates, using the downtime ledger from prior cycles as a live constraint input. This is what makes the fairness-over-time and max-consecutive-downtime constraints real rather than theoretical, and it's the architectural piece that separates this from a homework-scale CSP demo.
5. An LLM explanation layer that is read-only and evidence-grounded. It answers operator queries ("why was Sector 12 cut before Sector 9?") strictly by querying the solver's constraint trace and the relaxation log — it never sees or influences the decision. We will demonstrate this boundary explicitly (e.g., by showing the LLM cannot produce an answer that isn't traceable to a logged constraint), which is the actual interpretability contribution, not just "we added a chat interface."

What we are explicitly not claiming: we are not predicting shortfalls (that's a separate, well-studied forecasting problem we treat as an input), and we are not building a production-grade SCADA integration — the actuation layer is a realistic simulation with confirmable success/failure states, not a live grid connection.

---

5. System Architecture



Control loop, stated explicitly (this is the part the original diagram omits entirely): the system doesn't run once. It replans on a fixed cadence (e.g., every 5–15 simulated minutes) or whenever the shortfall changes by more than a threshold. Each replan reads the current downtime ledger from the Grid State Store, so a feeder that was just cut isn't immediately eligible to be cut again unless the constraint allows it — this is what actually enforces the fairness-over-a-rolling-window requirement, and it can't be enforced by a single-shot pipeline.

Agent framing (PEAS):

- Performance measure: minimize weighted harm (disruption × priority × duration) subject to fairness and safety constraints
- Environment: the grid's current state, live shortfall size, feeder history
- Actuators: issue cut/restore commands per feeder per time slot
- Sensors: shortfall magnitude, feeder states, elapsed downtime per feeder

---

6. Technical Approach — Mapped to the AI Course Syllabus

Revised from the original mapping, which was written against a single-shot pipeline and undersold two of our own core contributions (the rolling-horizon loop and the relaxation cascade). Both are placed explicitly below, and the A* vs. CSP comparison is pinned down rather than left ambiguous.

State-space search (Unit 2).

We fix an ambiguity the original diagram left implicit: A* and the CSP solvers are not searching the same space, and we say so rather than implying a fair head-to-head. A* runs on a deliberately simplified single-time-slot formulation — state = which feeders are currently on/off, transitions = toggle one feeder, goal = shortfall resolved for this slot only, heuristic = remaining shortfall ÷ largest available sheddable load (admissible, since no single toggle can shed more than the largest available load). This gives a clean informed-vs-uninformed search comparison and a sanity-check lower bound. It is not a substitute for the CSP, which reasons over the full feeder × time-slot space at once — duration, max-consecutive-downtime, and rolling-window fairness are not even expressible in a single-slot formulation. The write-up reports A* results as "what greedy per-slot search gets you, and what you lose by not reasoning across time slots jointly," never as a runtime-comparable alternative to the CSP solvers.

Constraint Satisfaction Problems (Unit 3) — the core of the project.

- Variables: state(feeder_i, time_slot_j) ∈ {on, off}
- Constraints: shortfall resolution per slot, hard-protected loads, max consecutive downtime, fairness bound over a rolling window
- Solvers to implement and compare: (a) backtracking search with forward checking / arc-consistency (AC-3), (b) min-conflicts local search, (c) a simple greedy baseline for contrast
- Deliverable: a runtime and solution-quality comparison table across increasing feeder counts (e.g., 10, 50, 200 feeders) to show where each approach breaks down
- The constraint-relaxation cascade (Section 4, deliverable #2) is itself CSP content, not an add-on: it's a meta-level policy over the same constraint set — deciding which constraints are soft, in what priority order they may be dropped, and proving the protected-feeder constraint is never among them. That ordering logic is exercised and tested the same way the core solvers are.

Knowledge Representation and Reasoning (Unit 4).

Encode protected-load rules, regulatory constraints, and priority classes as an explicit, inspectable rule base (not hardcoded if/else logic) — e.g., a small Horn-clause or production-rule representation the CSP layer queries when building constraints. The relaxation-order policy (fairness → max-downtime → priority class, protected-feeder rule never touched) lives in this same rule base as a versioned,

declarative artifact rather than a hardcoded branch in the solver — which is what makes it inspectable and arguable in a viva rather than just "trust the code."

Intelligent Agents & Expert Systems (Unit 5).

Frame the entire scheduler as an agent under the PEAS model (Section 5), and treat the rule base + constraint layer together as a lightweight expert system for load-shedding policy — this connects two units of the syllabus (agents and expert systems) into one coherent component rather than treating them as separate lecture topics. The rolling-horizon replanning loop belongs here explicitly: it is the sense–plan–act cycle that makes this an agent rather than a one-shot solver — the system perceives (shortfall + downtime ledger), plans (CSP solve, with relaxation if needed), acts (issues cut/restore commands), and re-senses on the next cadence tick. Presenting it this way also gives us a natural place to discuss a real agent-design failure mode (plan thrashing across replan cycles — see Sections 9 and 10) rather than treating the agent loop as a diagram-only formality.

Generative AI Fundamentals through LLMs (Units 6–9).

The only generative component is the explanation layer: given the computed schedule and the solver's constraint-satisfaction trace, an LLM answers operator questions in natural language by grounding its answer in the actual trace data (retrieved, not hallucinated) — a real, scoped use of prompt engineering and context-window management, rather than a bolt-on chatbot.

Responsible AI (Unit 10).

Fairness is a first-class, measured output (Section 4, deliverable #3) — report the fairness metric across simulated neighbourhoods for each solving strategy, and discuss explicitly where a purely cost-minimizing schedule would be unfair even while being "optimal," which is a real and gradable Responsible AI discussion point rather than a token paragraph.

---

7. Data and Simulation Plan

Real discom-level operational data (which feeders were cut, when, and why) is generally not public, so the plan combines confirmed public demand/shortfall sources with a synthetic-but-grounded grid topology. Confirmed sources (full field lists, access details, and license terms live in the companion data-sources.md, kept in sync with this section):

- Grid-India / POSOCO real-time and historical grid operation data — state-wise demand, generation, and frequency data published by the national grid operator; used for realistic demand magnitude and diurnal/seasonal load shape.
- A Kaggle-hosted multi-state demand/shortage compilation — used to check that our synthetic mild/moderate/crisis shortfall scenarios sit in a realistic range rather than being picked arbitrarily.
- An hourly state/regional load dataset — used for time-of-day load-curve shape when generating per-feeder synthetic loads.
- data.gov.in — secondary source for state electricity board statistics, used to cross-check the above rather than as a primary input.

None of these datasets contain feeder-level cut/restore decisions — that data is not public, as noted above. They inform demand magnitude and shortfall realism, not the scheduling decision itself. The grid topology, feeder count, priority classes, and protected-load tagging remain synthetic, constructed to

match publicly documented distribution-network structures and clearly labeled as synthetic in the final report — not claimed as real deployment data.

The scheduler is run under simulated shortfall scenarios of varying severity (mild, moderate, crisis-level), generated from the sources above, to stress-test constraint satisfiability and fairness under increasing pressure. Being upfront that the topology is synthetic-but-grounded is an academic-integrity point and will read as more credible than overclaiming real deployment.

---

8. Implementation Plan and Timeline

Revised to fix the structural gap in the original plan: every phase now has a named owner and a concrete, checkable exit criterion, not just a one-line deliverable. Team of three — Person A leads CSP solver implementation, Person B leads architecture/simulation/replanning, Person C leads literature, data, and evaluation — with cross-support noted where a phase leans on more than one person. This reshapes the original 9 rows into 11, since the rolling-horizon loop and the relaxation cascade are now full phases in their own right rather than folded into "fairness metrics."

Phase	Weeks	Owner	Deliverable	Exit Criteria
1. Problem formalization & literature review	1–2	Person C (all sign off)	Formal CSP definition; related-work table (industry + academic CSP-scheduling lineage); dataset selection	Formal spec reviewed and approved by all 3; data-sources.md lists ≥4 verified, accessible sources
2. Grid simulation & data pipeline	3–4	Person B	Synthetic grid topology generator; demand/shortfall scenario generator	Generator produces valid, protected-feeder-tagged topologies at 10/50/200/500 feeders; scenarios validated against Phase 1 data ranges
3. CSP solver — baseline (backtracking + AC-3)	5–6	Person A	Solver working end-to-end on small grids	100% constraint satisfaction at ≤50 feeders on all test scenarios; solve times logged
4. CSP solver — local search + comparison	7–8	Person A (B support)	Min-conflicts solver, greedy baseline, first runtime/quality comparison	Comparison table (solve time + harm-cost) complete across 10/50/200 feeders for all 3 solvers
5. Rolling-horizon replanning loop	9	Person B	Replan loop wired to live downtime ledger, cadence + threshold triggers	Replans correctly on both cadence and shortfall-delta threshold; passes a scripted multi-cycle test sequence
6. Constraint-relaxation cascade	10	Person A	Relaxation policy implemented and logged	Relaxation fires only when infeasible; order verified (fairness → downtime →

Phase	Weeks	Owner	Deliverable	Exit Criteria
				priority); protected-feeder rule never relaxed in any test case; every relaxation logged with reason
7. A* baseline & fairness metric	11	Person C	A* single-slot implementation + scope note; fairness metric wired into replan loop	A* results labeled explicitly as non-comparable per Section 6; Gini/max-to-mean metric logged automatically every replan cycle
8. Evaluation harness: scale, thrashing, relaxation	12	Person C (A + B support)	Scalability sweep; replanning-stability (thrashing) test; relaxation frequency/severity report	Sweep run at 10→500 feeders; thrashing rate measured and reported per scenario tier; relaxation frequency/severity reported per scenario tier
9. LLM explanation layer	13	Person B	Natural-language Q&A grounded in constraint trace	LLM answers verified against trace for a sample query set; boundary test shows it cannot answer beyond the logged trace
10. Scaling analysis & report writing	14	All	Full comparative write-up; scaling limits documented	Draft report covers all 4 deliverables with data from Phases 3–9; reviewed by all 3
11. Buffer, polish, demo prep	15	All	GitHub repo finalized, presentation, viva prep	Repo runs clean from a fresh clone; presentation rehearsed; each member can defend any section in viva

This assumes a 15-week semester; adjust proportionally to your actual course calendar. If time gets tight, cut the LLM explanation layer (Phase 9) first — the CSP comparison, relaxation cascade, and fairness analysis are where the AI-course grading rubric's weight actually sits (CSP + KR&R + agents), and the LLM layer is a differentiator, not the spine.

---

9. Evaluation Methodology

- Correctness: every produced schedule must be validated against all hard constraints (protected loads, shortfall resolution) — report a 100% constraint-satisfaction rate as a baseline sanity check, not a headline result.
- Solution quality: compare harm-cost across solving strategies for identical scenarios.
- Fairness: report the chosen fairness metric across strategies and scenarios; explicitly show cases where the "cheapest" schedule is also the least fair, to make the tradeoff visible.

- Scalability: runtime and solution quality as feeder count increases (10 → 50 → 200 → 500), to identify where backtracking degrades and where local search remains viable.
- Relaxation-cascade frequency and severity: for each scenario tier (mild/moderate/crisis), report how often the relaxation cascade fires, which constraint tier gets relaxed, and by how much (e.g., how far the schedule falls below the fairness bound when fairness is relaxed). This is now a first-class contribution (Section 4, deliverable #2) and needs its own reported metric, not a passing mention.
- Replanning stability: measure how often a feeder's on/off state flips between consecutive replan cycles with no corresponding change in shortfall or downtime ledger (plan thrashing) — a documented failure mode in rolling-horizon / MPC-style controllers. Report a thrashing rate per scenario tier and check whether it clusters around planning-window boundaries.
- Explanation quality (LLM layer, if built): manually verify a sample of explanations against the actual constraint trace to confirm the LLM is not hallucinating justifications.

---

10. Risks and Honest Limitations

- Synthetic data risk: results are only as credible as the grid topology's realism — mitigate by grounding parameters in public discom documentation and stating assumptions explicitly.
- CSP scalability wall: backtracking-based exact solving will likely become impractical well before 500+ feeders — this is expected and should be reported as a finding, not hidden.
- Replanning instability (plan thrashing): rolling-horizon replanning is a new architectural decision with its own failure mode — a feeder can be toggled on then off across consecutive replan cycles due to boundary effects between planning windows, even when the underlying shortfall hasn't meaningfully changed. Mitigate with hysteresis (e.g., a minimum-state-duration constraint on toggles) and measure it directly via the thrashing metric in Section 9, rather than discovering it during the Week 12 evaluation pass.
- Scope creep on the LLM layer: it's tempting to expand the LLM component into a bigger feature; resist this — its job is explanation, not decision-making, and expanding its role would undercut the project's core CSP/search contribution.
- Fairness metric choice is a modeling decision, not a neutral fact — be prepared to justify why you chose Gini coefficient (or whichever metric) over alternatives in the viva.

---

11. Possible Extensions (Stretch Goals, Not Core Scope)

- Incorporate a real (even simple) forecasting layer from the ML course as a pluggable input, demonstrating the two-course thematic connection without hard-coupling deadlines.
- Multi-region coordination: extend the CSP to reason across multiple interconnected grid zones with shared shortfall constraints.
- Reinforcement-learning-based policy as an additional comparison point against the CSP solvers, directly using the ML course's intro-RL unit.